

Министерство науки и высшего образования Российской Федерации
Московский физико-технический институт (государственный университет)

Физтех-школа радиотехники и компьютерных технологий
Кафедра микропроцессорных технологий в интеллектуальных системах управления

Выпускная квалификационная работа бакалавра

Реализация векторизации ациклических участков кода по методу Superword-Level Parallelism в компиляторе языка программирования Go

Автор:

Студент Б01-2076 группы
Самойлов Арсений Сергеевич

Научный руководитель:

научная степень

Петушков Игорь Вячеславович

Научный консультант:

Маркин Алексей Леонидович



Москва 2026

Аннотация

Реализация векторизации ациклических участков кода по методу Superword-Level Parallelism в компиляторе языка программирования Go
Самойлов Арсений Сергеевич

В работе рассматривается задача автоматической векторизации программного кода в компиляторе языка Go. Отсутствие в стандартном компиляторе `cmd/compile` оптимизаций, использующих SIMD-инструкции, ограничивает производительность вычислительно насыщенных приложений. Существующие реализации на базе LLVM (например, `gollvm`) неприемлемы из-за высоких накладных расходов на компиляцию и отставания от основного компилятора. В работе предложен и реализован проход SLP-векторизации, адаптированный к SSA-представлению и архитектурным ограничениям Go: явному моделированию памяти, отсутствию сложных анализов и требованию быстрой компиляции. Разработанный векторизатор выполняет поиск изоморфных независимых операций, расширение групп, валидацию и генерацию 128-битных векторных инструкций для архитектур x86-64 (SSE/AVX) ARM64 (NEON). Экспериментальная оценка на синтетических тестах и реальных приложениях (трассировщик лучей, `etcd`, набор `Sweet`) показала ускорение до XX% на вычислительно интенсивных фрагментах при незначительном увеличении времени компиляции (менее 1%). Работа демонстрирует принципиальную возможность и практическую пользу внедрения SLP-векторизации в основной компилятор Go.

Содержание

1	Введение	5
2	Постановка задачи	6
3	Автоматическая векторизация программного кода	7
3.1	SIMD как аппаратная основа векторизации	7
3.2	Подходы к автоматической векторизации	8
3.2.1	Цикловая векторизация	8
3.2.2	SLP-векторизация	8
3.2.3	Сравнение подходов	9
3.3	Метод Superword-Level Parallelism	9
3.3.1	Основные этапы алгоритма	9
3.3.2	Преимущества метода SLP	10
3.3.3	Ограничения метода	10
3.3.4	Причины выбора SLP для компилятора Go	10
3.4	Выводы по главе	11
4	Обзор существующих решений	12
4.1	Реализация в LLVM	12
4.1.1	Архитектура раннего SLP-векторизатора LLVM	13
4.1.2	Построение и оценка дерева выражений	13
4.1.3	Генерация векторного кода	13
4.1.4	Основные ограничения ранней реализации	13
4.1.5	Значение для выполненной работы	14
4.2	Реализация в GCC	14
4.3	Ограничения переноса существующих решений в компилятор Go	14
4.3.1	Ограниченный набор аналитических проходов	14
4.3.2	Явное моделирование памяти в Go SSA	15
4.3.3	Философия быстрой компиляции	15
4.3.4	Итог	15
5	Компилятор Go	16
5.1	Существующие реализации компилятора Go и проблема векторизации	16
5.2	Общая структура компилятора Go	16
5.3	SSA-представление	17
5.4	Оптимизационный конвейер	18
5.5	Особенности SSA, значимые для SLP-векторизации	18
5.6	Архитектурные ограничения для внедрения SLP	19
5.6.1	Явное моделирование памяти	19
5.6.2	Ограниченная аналитическая инфраструктура	20
5.6.3	Требования к времени компиляции	20
5.6.4	Особенности рантайма языка	20
5.7	Выбор точки интеграции	21
5.8	Выводы по главе	21
6	Проектирование SLP-векторизатора	23
6.1	Требования и ограничения для Go-векторизатора	23
6.2	Общая архитектура	23
6.3	Поиск кандидатов для векторизации	24

6.4	Построение графа зависимостей и расширение групп	25
6.5	Валидация и планирование	25
6.5.1	Проверка внешних использований	25
6.5.2	Проверка замкнутости операндов	26
6.5.3	Проверка когерентности линий	26
6.5.4	Проверка исходных аргументов	26
6.6	Легализация и генерация кода	26
6.7	Принятые компромиссы и направления развития	27
7	Реализация	28
7.1	Инструментарий и среда разработки	28
7.2	Базовые структуры данных и проверки	28
7.3	Проход <code>slp</code> : главный управляющий цикл	29
7.4	Этапы анализа	29
7.4.1	Поиск начальных пар	29
7.4.2	Расширение групп	29
7.4.3	Комбинирование	29
7.4.4	Разбиение на компоненты связности	30
7.5	Валидация и планирование	30
7.5.1	Валидация	30
7.5.2	Планирование	30
7.6	Генерация векторного SSA-кода	30
7.6.1	Легализация	30
7.6.2	Создание векторных инструкций	30
7.6.3	Замена скалярных использований	31
7.7	Тестирование корректности	31
8	Экспериментальная оценка реализации	32
8.1	Цели и методика оценки	32
8.2	Выбор тестовых наборов	32
8.2.1	Набор синтетических тестов	32
8.2.2	Трассировщик лучей	33
8.2.3	<code>etcd</code>	33
8.2.4	<code>Sweet</code>	33
8.3	Результаты измерения производительности	33
8.3.1	Набор синтетических тестов	33
8.3.2	Трассировщик лучей	34
8.3.3	<code>etcd</code>	34
8.3.4	<code>Sweet</code>	34
8.4	Влияние на время компиляции	34
8.5	Сравнение с <code>gollvm</code>	34
8.6	Обсуждение ограничений и не векторизованных случаев	34
8.7	Направления будущей работы	35
9	Заключение	36

1 Введение

Современные программные системы нуждаются в большем количестве вычислений. Особенно остро данный вопрос стоит для серверных приложений, обрабатывающих значительные объёмы данных. Повышение производительности программного обеспечения позволяет эффективнее использовать аппаратные ресурсы, снижая потребность в дополнительном серверном оборудовании. Даже относительно небольшое ускорение может приводить к заметному экономическому эффекту в масштабах крупных систем. Одним из важных аппаратных механизмов повышения производительности современных процессоров является SIMD-параллелизм (Single Instruction Multiple Data), позволяющий выполнять за одну инструкцию операцию одновременно над несколькими элементами данных. Несмотря на широкую аппаратную поддержку SIMD-инструкций в современных процессорных архитектурах, ручное написание векторизованного кода требует глубокого знания особенностей конкретной архитектуры, что усложняет сопровождение программ и существенно снижает их переносимость. В связи с этим особую практическую значимость имеют методы автоматической векторизации, позволяющие компилятору самостоятельно преобразовывать скалярный код в его векторный эквивалент. Особый интерес задача автоматической векторизации представляет для языка программирования Go. Данный язык широко применяется при разработке высоконагруженных серверных систем. Однако автоматическая векторизация в главной реализации компилятора Go отсутствует. Это приводит к недоиспользованию аппаратных возможностей современных процессоров и снижает потенциальную производительность программ. Среди существующих подходов к автоматической векторизации важное место занимает метод Superword-Level Parallelism (SLP). В отличие от цикловой векторизации, ориентированной на анализ и преобразование циклических конструкций, SLP-векторизация направлена на поиск независимых однотипных операций внутри ациклических участков программы. Такой подход требует менее сложного анализа зависимостей, проще интегрируется в компилятор и обладает меньшими накладными расходами на этапе компиляции. Выбор метода SLP особенно актуален для компилятора Go, так как реализация сложных цикловых оптимизаций потребовала бы существенного расширения существующей инфраструктуры анализа, тогда как SLP-подход позволяет достичь практического ускорения на ряде классов задач при умеренной сложности реализации. Сложность алгоритма особенно важна, поскольку одной из ключевых характеристик для разработчиков на Go является время компиляции.

2 Постановка задачи

Объектом исследования является подсистема SSA-оптимизаций (SSA — Static Single Assignment форма представления инструкций) компилятора языка Go.

Предметом исследования являются методы автоматической векторизации ациклических участков кода на основе подхода Superword-Level Parallelism.

Целью работы является разработка и реализация прототипа механизма автоматической SLP-векторизации ациклических участков кода в компиляторе Go.

Для достижения поставленной цели в работе решаются следующие задачи:

- исследовать существующие методы автоматической векторизации;
- проанализировать архитектуру оптимизационного конвейера компилятора Go;
- разработать алгоритм выявления векторизуемых групп операций;
- реализовать преобразования SSA-представления для генерации векторных операций;
- провести экспериментальную оценку корректности и эффективности разработанного решения.

Практическая значимость работы заключается в создании прототипа механизма автоматической векторизации для компилятора Go, демонстрирующего возможность интеграции SLP-преобразований в существующий компилятор. Полученные результаты могут быть использованы в качестве основы для дальнейшего развития оптимизирующих возможностей компилятора.

3 Автоматическая векторизация программного кода

3.1 SIMD как аппаратная основа векторизации

SIMD представляет собой модель вычислений, при которой одна инструкция одновременно применяется к нескольким элементам данных, размещённым в специальном векторном регистре. Такой подход позволяет существенно увеличить вычислительную производительность при выполнении однотипных операций над независимыми данными.

Принцип работы SIMD можно проиллюстрировать на примере сложения двух массивов целых чисел. При скалярном выполнении операция сложения производится последовательно для каждой пары элементов:

$$c_i = a_i + b_i, \quad i = 0, \dots, n-1$$

При использовании SIMD несколько элементов обрабатываются одновременно одной инструкцией:

$$c_{[i,i+m]} = a_{[i,i+m]} + b_{[i,i+m]}, \quad i = 0, \dots, n/m - 1,$$

где m — количество данных, помещающихся в векторный регистр. Если ширина векторного регистра составляет 128 бит, то за одну инструкцию можно выполнить сложение двух 64-битных целых чисел ($m = 2$) или четырех 32-битных ($m = 4$).

Современные процессорные архитектуры предоставляют развитую поддержку SIMD-вычислений. В архитектуре ARM данная функциональность предоставляется расширениями NEON и SVE, отличающимися шириной векторных регистров и набором поддерживаемых операций. В архитектуре x86 аналогичные возможности реализованы через наборы инструкций SSE, AVX, AVX2 и AVX-512.

Использование SIMD особенно эффективно в задачах:

- обработки массивов числовых данных;
- цифровой обработки сигналов;
- криптографических вычислениях;
- мультимедийной обработки;
- математических вычислений.

Однако эффективное применение SIMD-инструкций предъявляет ряд требований к структуре программы. Операции должны обладать независимостью по данным, иметь совместимый тип и размер операндов, а также допускать группировку в векторные операции.

Кроме того, непосредственное использование SIMD-инструкций связано с рядом практических трудностей. Разработчику необходимо учитывать особенности конкретной аппаратной архитектуры, различия в наборах инструкций и специфику работы с векторными регистрами. Это существенно усложняет разработку и снижает переносимость программного обеспечения.

По этой причине особую важность приобретают механизмы автоматической векторизации, позволяющие компилятору самостоятельно выявлять участки кода, пригодные для преобразования в векторную форму.

3.2 Подходы к автоматической векторизации

Автоматическая векторизация представляет собой класс оптимизаций компилятора, направленных на преобразование скалярного программного кода в эквивалентную форму, использующую SIMD-инструкции.

В современных компиляторах можно выделить два основных подхода к автоматической векторизации:

- цикловая векторизация;
- SLP-векторизация.

3.2.1 Цикловая векторизация

Цикловая векторизация ориентирована на поиск параллелизма внутри циклов. Рассмотрим следующий пример:

$$\text{for } i := 0; i < n; i++ \{ c[i] = a[i] + b[i] \}$$

Такой цикл содержит естественный параллелизм по итерациям. Компилятор может преобразовать его таким образом, чтобы за одну итерацию выполнялась обработка сразу нескольких элементов.

Эффективность цикловой векторизации во многом определяется возможностью доказать отсутствие зависимостей между итерациями цикла. Для этого компилятору необходимо выполнять сложный статический анализ, включающий:

- анализ псевдонимов памяти (alias analysis);
- анализ межитерационных зависимостей (loop-carried dependency analysis);

а также ряд вспомогательных преобразований, таких как:

- разворачивание цикла (loop unrolling);
- выделение граничных итераций (loop peeling).

Реализация подобных механизмов требует развитой инфраструктуры анализа и существенно усложняет оптимизационный конвейер компилятора.

3.2.2 SLP-векторизация

SLP-векторизация ориентирована на поиск параллелизма внутри ациклических участков программы. В отличие от цикловой векторизации, данный подход не анализирует зависимости между итерациями, а выявляет независимые однотипные скалярные операции, которые могут быть объединены в векторную инструкцию.

Например, последовательность:

$$x[0] = a[0] + b[0], \quad x[1] = a[1] + b[1]$$

может быть преобразована в одну векторную операцию сложения.

SLP-векторизация особенно эффективна в случаях, когда параллелизм явно присутствует в структуре вычислений, но не выражен через циклические конструкции.

3.2.3 Сравнение подходов

Цикловая и SLP-векторизация существенно различаются по сложности реализации. Цикловая векторизация требует сложного анализа, но позволяет оптимизировать циклические участки кода, которые зачастую являются основной вычислительной нагрузкой большого класса приложений. SLP-векторизация обладает меньшей вычислительной сложностью и проще интегрируется в инфраструктуру компилятора.

Для компилятора Go, где отсутствуют циклические оптимизации и нужная для них инфраструктура анализа, а также где одним из ключевых требований является быстрое время компиляции, SLP-подход представляет особый практический интерес.

3.3 Метод Superword-Level Parallelism

Метод Superword-Level Parallelism был предложен в работе Samuel Larsen и Saman Amarasinghe [1] как способ выявления параллелизма на уровне независимых скалярных операций внутри базовых блоков программы. Основная идея метода заключается в объединении нескольких однотипных независимых скалярных операций в одну векторную.

3.3.1 Основные этапы алгоритма

Классический алгоритм SLP-векторизации [1] состоит из нескольких последовательных этапов, выполняемых в пределах одного базового блока.

1. **Поиск начальных пар.** Алгоритм сканирует инструкции базового блока и ищет пары соседних обращений к памяти – смежные загрузки или сохранения данных. Такие операции могут послужить естественным началом и/или завершением параллельных последовательностей вычислений.
2. **Расширение групп.** От найденных начальных пар алгоритм итеративно строит новые группы, анализируя поток данных:
 - *Расширение вверх:* для каждой пары инструкций в группе анализируются их аргументы – если находятся две однотипные независимые операции, они могут образовать новую пару.
 - *Расширение вниз:* рассматриваются операции, использующие результат исходных операций. Среди нескольких возможных вариантов выбирается та пара использований, которая приносит наибольшую выгоду с точки зрения модели стоимости.

Процесс продолжается, пока находятся новые пары, удовлетворяющие условиям независимости и изоморфности операций.

3. **Объединение пар в группы.** После завершения расширения ищутся пары, которые можно объединить. Если в двух парах есть общая операция, то их можно объединить в тройки, четвёрки и т.д., формируя потенциальные векторные регистры большей ширины.
4. **Планирование.** Векторные инструкции должны располагаться в корректном порядке с учётом зависимостей по данным. Выполняется топологическая сортировка групп: ни одна группа не может быть помещена в выходной поток раньше тех групп, от которых она зависит через аргументы.

5. **Генерация векторного кода.** Для каждой группы формируются соответствующие векторные инструкции целевой архитектуры. При необходимости группы разбиваются до поддерживаемой аппаратурой ширины. Если какая-то часть промежуточного результата используется вне векторизованного фрагмента, то нужное значение скаляризуется – вставляются специальные операции извлечения отдельных данных из векторного регистра в обычный.

Такой набор этапов гарантирует корректность векторизации и позволяет последовательно расширять функциональность, начиная с простых парных операций.

3.3.2 Преимущества метода SLP

Метод Superword-Level Parallelism обладает рядом существенных преимуществ.

- **Простота и низкие накладные расходы.** Алгоритм работает исключительно на уровне отдельных базовых блоков, что упрощает реализацию и снижает время компиляции.
- **Расширяемость.** Метод SLP допускает поэтапное наращивание функциональности: от поддержки простейших целочисленных операций до межблочной векторизации. Более того, SLP может быть расширен в сторону учёта циклических конструкций. Примером служит Loop-Aware SLP, реализованный в GCC [?], который анализирует возможности векторизации инструкций через границы итераций цикла, объединяя преимущества классического SLP и цикловой векторизации.

Перечисленные свойства делают SLP прагматичным компромиссом между сложностью, временем компиляции и достигаемым ускорением.

3.3.3 Ограничения метода

Несмотря на преимущества, SLP обладает рядом ограничений.

- Метод эффективен только при наличии явно выраженных групп независимых однотипных операций. Если вычисления имеют сложную структуру зависимостей или требуют существенного переупорядочивания данных, эффективность SLP снижается.
- Алгоритм чувствителен к качеству модели стоимости. Слишком агрессивная векторизация может привести к увеличению числа вспомогательных операций упаковки/распаковки и ухудшению производительности.

3.3.4 Причины выбора SLP для компилятора Go

Выбор метода SLP для реализации в компиляторе Go обусловлен рядом факторов. Данный подход хорошо согласуется с существующей архитектурой компилятора. Реализация SLP требует существенно меньшего объёма инфраструктурных изменений по сравнению с цикловой векторизацией. Алгоритм обладает умеренной вычислительной сложностью, что соответствует требованию сохранения малого времени компиляции.

Таким образом, SLP представляет собой практически обоснованный компромисс между сложностью реализации и потенциальным приростом производительности.

3.4 Выводы по главе

В данной главе были рассмотрены аппаратные основы SIMD-вычислений, основные подходы к автоматической векторизации и детали метода Superword-Level Parallelism. Показано, что SLP-векторизация, в отличие от цикловой, не требует анализа итерационных зависимостей и сложных трансформаций циклов. Это делает её практически применимым подходом для интеграции в компилятор Go: умеренная алгоритмическая сложность сочетается с возможностью ускорять ациклический код, типичный для многих серверных и системных программ на Go.

4 Обзор существующих решений

Метод SLP получил широкое распространение в современных оптимизирующих компиляторах. Наиболее известные промышленные реализации присутствуют в инфраструктурах LLVM и GCC. За время развития данных проектов алгоритмы SLP-векторизации существенно усложнились и были дополнены многочисленными эвристиками, ориентированными на повышение качества генерируемого кода и поддержку широкого спектра архитектур.

4.1 Реализация в LLVM

SLP-векторизатор был добавлен в LLVM в 2013 году. Первоначальная реализация представляла собой сравнительно компактный алгоритм, ориентированный на поиск групп изоморфных инструкций и построение деревьев векторизации внутри базовых блоков.

Основными возможностями ранней реализации являлись:

- объединение однотипных арифметических операций;
- векторизация последовательных загрузок и сохранений;
- базовая модель оценки прибыльности преобразований.

Дальнейшее развитие LLVM привело к существенному расширению функциональности SLP-векторизатора. Современная реализация поддерживает:

- более сложные схемы перестановки элементов;
- частичную векторизацию;
- сложные стратегии переупорядочивания операций.

В результате кодовая база современного SLP-векторизатора LLVM значительно выросла и включает большое количество эвристик, ориентированных на конкретные архитектурные особенности.

В рамках данной работы для анализа была рассмотрена ранняя реализация SLP-векторизации, представленная в LLVM версии 3.3. Выбор именно данной версии обусловлен несколькими причинами:

1. Ранняя реализация содержит базовые алгоритмические идеи метода в относительно компактной и понятной форме.
2. Она лишена значительного количества инженерных усложнений, накопленных в современных версиях LLVM.
3. Архитектурные решения раннего SLP-векторизатора лучше подходят для адаптации в условиях ограниченной инфраструктуры анализа, характерной для компилятора Go.

Таким образом, использование реализации LLVM 3.3 для анализа позволяет опираться на проверенный промышленный подход, сохраняя при этом приемлемую сложность.

4.1.1 Архитектура раннего SLP-векторизатора LLVM

В основе алгоритма лежит обход базового блока, начинающийся с поиска цепочек последовательных инструкций сохранения. Для этого используется анализатор скалярной эволюции (ScalarEvolution [?]), определяющий, обращаются ли две инструкции к смежным областям памяти. Если две записи сохраняют однотипные значения по адресам, отличающимся ровно на размер элемента, они считаются последовательными.

Найденные пары объединяются в цепочки: пока следующее сохранение является смежным с предыдущим и ещё не помечено как векторизованное, оно добавляется в группу. Далее для каждой цепочки запускается попытка векторизации скользящим окном, размер которого определяется целевой шириной векторного регистра (по умолчанию 128 бит).

4.1.2 Построение и оценка дерева выражений

Для выбранного окна сохранений строится дерево выражений путём рекурсивного подъёма по аргументам. Алгоритм поднимается к операциям, породившим сохраняемые значения. Рекурсивно формируется набор инструкций-предшественников, проверяется их изоморфизм и допустимость упаковки.

Процесс управляется классом `BoUpSLP`, который на этапе анализа стоимости просматривает дерево с ограничением глубины. Все инструкции нумеруются для контроля порядка, а также фиксируется, в какой «линии» (позиции внутри будущего вектора) используется каждое значение. Если одна и та же инструкция оказывается востребованной в разных линиях, она помечается как подлежащая обязательной скаляризации — её результат не может быть частью векторного регистра без дублирования, что обычно дорого.

Стоимость векторизации рассчитывается рекурсивно: для каждого узла дерева стоимость скалярных эквивалентов сравнивается со стоимостью векторной операции, получаемой через интерфейс `TargetTransformInfo`. Учитываются затраты на загрузку/сохранение, арифметические операции и возможные вставки/извлечения элементов. Дополнительно проверяется, не мешают ли промежуточные инструкции, работающие с памятью, безопасному перемещению ранних нагрузок к месту размещения векторизованных операций (анализ `isUnsafeToSink`).

4.1.3 Генерация векторного кода

Если модель стоимости показывает выгоду, запускается `vectorizeTree`. Рекурсивно, начиная с листьев дерева (обычно загрузок или констант) и двигаясь к корню, создаются векторные инструкции.

Для случаев, когда дерево не завершается сохранением, существует метод `vectorizeArithmetic`, который строит вектор, а затем вставляет операции извлечения элементов для каждого исходного использования, сохраняя корректность программы.

4.1.4 Основные ограничения ранней реализации

Ранний SLP-векторизатор LLVM обладает рядом естественных ограничений, многие из которых были сняты в более поздних версиях:

- **Один базовый блок:** векторизация не переходит через границы блоков; РН-узлы не поддерживаются.

- **Фиксированная векторная ширина:** используется минимальный размер векторного регистра (128 бит), кратность ширине скалярного типа. Нет поддержки частичной упаковки неполных векторов.
- **Жёсткий анализ последовательности:** функция `isConsecutiveAccess` работает только с константным расстоянием между адресами, не поддерживая индексацию с переменным шагом.
- **Глубина рекурсии:** ограничение в несколько уровней рекурсии не позволяет векторизовать глубокие деревья выражений.
- **Правило однократного использования:** если скалярная инструкция имеет несколько использований, она помечается `MustScalarize`, что часто делает векторизацию невыгодной.

4.1.5 Значение для выполненной работы

Ранняя реализация LLVM примечательна тем, что в сжатой форме воплощает ключевые принципы SLP: поиск начальных пар через память, рекурсивное расширение, оценку стоимости и генерацию векторных инструкций. Её изучение позволяет выделить алгоритмическое ядро, свободное от инженерных наслоений поздних версий. Именно поэтому версия LLVM 3.3 выбрана в качестве основного референса для проектирования собственного прохода.

Вместе с тем, восходящий от сохранений подход плохо согласуется с устройством SSA-формы компилятора Go, где целостность памяти задана явным потоком значений `Memory`. Прямое следование логике LLVM потребовало бы существенной переработки, что выводится за рамки данного этапа и подробно обсуждается в разделе 4.3 (Ограничения переноса существующих решений в компилятор Go).

4.2 Реализация в GCC

В компиляторе GCC SLP-векторизация реализована как часть более общей инфраструктуры `loop` и `basic-block vectorization`. Подход GCC тесно интегрирован с развитой системой анализа циклов и зависимостей. Это позволяет эффективно сочетать цикловую и SLP-векторизацию, однако требует значительно более сложной инфраструктуры промежуточного представления и анализа.

Детальный анализ реализации GCC выходит за рамки данной работы, так как основным референсом выбран LLVM. При необходимости описание может быть добавлено на следующем этапе.

4.3 Ограничения переноса существующих решений в компилятор Go

Несмотря на наличие работоспособных и проверенных временем реализаций SLP-векторизации в таких компиляторах, как LLVM и GCC, их прямое или близкое к исходному заимствование в компиляторе Go оказывается невозможным. Причины лежат в фундаментальных различиях архитектуры промежуточного представления, состава аналитической инфраструктуры и целевых установок проекта.

4.3.1 Ограниченный набор аналитических проходов

Компиляторы LLVM и GCC опираются на развитые анализы: точный анализ псевдонимов указателей (`aliasing`), скалярную эволюцию, анализ зависимостей по данным, анализ циклов. Ранний SLP-векторизатор LLVM активно использует `ScalarEvolution`

для проверки смежности адресов и `AliasAnalysis` для оценки безопасности перемещения инструкций памяти.

Инфраструктура компилятора Go, напротив, сознательно минимизирована: полноценный анализ псевдонимов отсутствует, единственным источником сведений о смежности адресов являются проходы `memcombine` и `pair`, работающие с простыми локальными шаблонами. Перенос даже ранней версии LLVM потребовал бы либо воссоздания значительной части аналитической подсистемы, либо замены точных проверок консервативными эвристиками.

4.3.2 Явное моделирование памяти в Go SSA

В LLVM IR порядок операций с памятью не задан жёстко; две независимые загрузки из заведомо различных областей могут быть переставлены, а `store`-инструкции не связываются виртуальными рёбрами. В Go SSA каждая операция, затрагивающая память, явно принимает и порождает значение типа `Memory`, образуя упорядоченную цепочку состояний. Эта модель, удобная для потокового анализа и понижения до машинного кода, становится серьёзным препятствием для восходящей стратегии SLP, принятой в LLVM. Две потенциально векторизуемые загрузки могут иметь различные входные значения `Memory` из-за наличия между ними записи по гарантированно не пересекающемуся адресу, что с точки зрения консервативного анализа делает их зависимыми и не подлежащими упаковке.

4.3.3 Философия быстрой компиляции

Компилятор Go проектировался с приоритетом скорости сборки, поэтому в нём не приветствуются дорогостоящие анализы, способные замедлить трансляцию крупных проектов. Ранний SLP-векторизатор LLVM, при всей своей относительной простоте, уже включает рекурсивный обход дерева с квадратичными проверками и нетривиальной моделью стоимости. Прямой перенос подобного прохода без адаптации к ограничениям по времени компиляции противоречил бы проектным принципам Go, требуя аккуратного выбора алгоритмических компромиссов.

4.3.4 Итог

Перечисленные факторы в совокупности делают невозможным простое портирование существующей реализации SLP-векторизатора. Вместо этого требуется самостоятельная разработка, опирающаяся на базовые идеи метода, но адаптированная к модели памяти, инфраструктуре анализа и ограничениям для быстрой компиляции, присущим компилятору Go.

5 Компилятор Go

5.1 Существующие реализации компилятора Go и проблема векторизации

Основным (и официально поддерживаемым) компилятором языка Go является `cmd/compile` [?] — самодостаточный компилятор, ориентированный на высокую скорость сборки. Именно он является основным в экосистеме Go. Однако существует и несколько альтернативных реализаций, среди которых наиболее заметна `gollvm` [?] — компилятор, построенный на инфраструктуре LLVM.

На первый взгляд, `gollvm` мог бы полностью снять необходимость разработки собственного векторизатора для Go: LLVM содержит зрелые и активно развиваемые проходы цикловой и SLP-векторизации, способные генерировать высокоэффективный SIMD-код. Тем не менее использование `gollvm` в качестве основного инструмента для внедрения автоматической векторизации в экосистему Go сопряжено с рядом фундаментальных проблем, делающих его неприемлемым на практике.

- **Несоответствие требованию быстрой компиляции.** LLVM предоставляет богатый набор глубоких анализов и агрессивных оптимизаций, но ценой значительного увеличения времени компиляции. Сборка `gollvm`’ом может занимать в разы или даже на порядок больше времени по сравнению со стандартным компилятором. Учитывая, что быстрота компиляции является одной из ключевых ценностей языка, переход на LLVM-бэкенд для массового использования маловероятен — это противоречит ожиданиям разработчиков.
- **Отставание от основного компилятора и нестабильность.** `gollvm` является экспериментальным проектом и не входит в официальный набор инструментов Go. Его развитие отстаёт от основной ветки: внедрение новых возможностей языка, изменений в спецификации или внутреннего устройства рантайма зачастую требует длительной адаптации.
- **Несовместимость с рантаймом Go.** Стандартный рантайм Go тесно завязан на конкретные свойства кода: точный сборщик мусора опирается на карты указателей, планировщик горутин ожидает определённой структуры стековых фреймов и корректного сохранения/восстановления регистров в точках синхронизации. Поэтому `gollvm` пользуется собственной версией рантайма, которая сильно уступает основной.

Вывод для настоящей работы. Таким образом, хотя `gollvm` демонстрирует потенциальные выгоды от векторизации Go-кода, он не может служить практическим решением для большинства пользователей языка. Единственным путём внедрения автоматической векторизации в Go является разработка соответствующих оптимизационных проходов непосредственно внутри стандартного компилятора `cmd/compile` с полным учётом его архитектуры, ограничений и требований рантайма. Именно это и составляет предмет данной работы. Далее в главе рассматривается структура и особенности стандартного компилятора Go, значимые для реализации SLP-векторизатора.

5.2 Общая структура компилятора Go

Компилятор языка программирования Go ориентирован на быстрое время компиляции и генерацию эффективного машинного кода для основных аппаратных архитектур (x86-64, ARM64, MIPS64 и др.). Реализация компилятора включает несколько основных этапов обработки исходного кода:

Placeholder for compiler pipeline diagram

Рис. 1: Основные этапы компиляции Go

1. **Синтаксический анализ и построение AST** — исходный текст преобразуется в абстрактное синтаксическое дерево, отражающее структуру программы.
2. **Проверка типов** — выполняется статическая типизация, вычисляются размеры типов, разрешаются идентификаторы.
3. **Преобразование в промежуточное представление** — типизированное AST транслируется в архитектурно-независимое SSA-представление.
4. **Фаза оптимизаций** — к SSA-форме применяется серия проходов (упрощение выражений, устранение мёртвого кода, устранение проверок границ, объединение обращений к памяти и др.).
5. **Генерация машинного кода** — финальное SSA-представление проходит распределение регистров и кодогенерацию, порождая объектный файл.

С точки зрения реализации оптимизаций наибольший интерес представляет фазы 4–5 компилятора, работающие с промежуточным представлением в форме SSA (Static Single Assignment). Использование SSA обеспечивает удобную основу для анализа зависимостей данных и локальных преобразований, что делает данное представление естественной точкой интеграции механизмов автоматической векторизации. В отличие от LLVM, Go-компилятор содержит компактный фиксированный набор оптимизационных проходов, каждый из которых выполняет строго определённую задачу, а конвейер сознательно упрощён ради предсказуемого времени компиляции.

5.3 SSA-представление

SSA-представление компилятора Go (пакет `cmd/compile/internal/ssa`) реализовано в виде набора связанных структур данных, ключевыми из которых являются `Block` (базовый блок) и `Value` (инструкция-значение). Каждая функция после построения SSA состоит из графа базовых блоков, внутри каждого из которых располагается линейный список значений. Значения нумеруются в порядке следования инструкций, что позволяет быстро сравнивать их позиции.

Основные свойства SSA-формы в Go:

- **Явные зависимости по данным** — каждое значение ссылается на свои аргументы. Это упрощает анализ потока данных.
- **Единый тип значения** — каждое значение имеет конкретный тип: указатели, целые числа, числа с плавающей точкой и составные типы.
- **Моделирование памяти** — операции, взаимодействующие с памятью (загрузки, сохранения, вызовы функций), принимают и возвращают специальное значение типа `TypeMem`, образуя цепочку состояний памяти. Это гарантирует корректный порядок операций.
- **Коды операций** — множество операций разделено на архитектурно-независимые и архитектурно-зависимые. В процессе компиляции обобщённые операции понижаются до машинно-зависимых.

- **Отсутствие данных для циклового анализа** — SSA-форма не содержит информации о циклах, индуктивных переменных или регионах; вся информация о структуре программы представлена графом базовых блоков.

Эти свойства делают SSA-представление удобным для реализации SLP-векторизации: зависимости данных прозрачны, типы известны статически, а память контролируется явными аргументами, позволяющими консервативно, но уверенно гарантировать отсутствие конфликтов.

5.4 Оптимизационный конвейер

Оптимизационный конвейер компилятора Go состоит из фиксированной последовательности проходов, выполняемых над SSA-представлением функции. К числу основных проходов относятся (в порядке их исполнения):

- **deadcode** — удаление мёртвого кода (удаление инструкций, результаты которых не используются).
- **prove** — устранение лишних проверок, в том числе проверок выходов за границы массивов.
- **memcombine** — объединение соседних узких загрузок/сохранений в широкие в тех случаях, когда адреса смежны и выровнены.
- **lower** — понижение до машинных операций (замена обобщённых Op-кодов на архитектурно-специфичные).
- **regalloc** — распределение регистров (замена виртуальных регистров физическими).

Проходы выполняются в строго определённом порядке, и каждый из них может изменять SSA-граф. Для внедрения SLP-векторизации важно выбрать такую точку интеграции, где базовые блоки не содержат машинно-зависимые операции, память представлена явно, а избыточные проверки устранены. Естественная позиция — до **lower**.

5.5 Особенности SSA, значимые для SLP-векторизации

Применительно к задаче SLP-векторизации Go SSA демонстрирует ряд особенностей, которые определяют архитектуру будущего прохода:

- **Явное кодирование зависимостей по памяти.** Каждая операция с памятью явно принимает и возвращает `TypeMem`, что позволяет консервативно проверять независимость инструкций: две нагрузки могут быть упакованы только в том случае, если они разделяют общее состояние памяти (или связаны допустимой цепочкой зависимостей).
- **Наличие информации о типах.** Для каждого значения известен его тип и размер. Это позволяет быстро фильтровать кандидатов по совместимости (одинаковая разрядность) и вычислять ширину будущего векторного значения.
- **Отсутствие сложных косвенных указателей.** Благодаря тому, что Go не поддерживает арифметику указателей общего назначения, анализ смежности адресов сводится к сравнению базовых указателей и константных смещений, что существенно проще общего случая C/C++.

- **Ограниченный набор подлежащих векторизации операций.** Только базовые арифметические, логические операции и обращения с памятью.
- **Активное развитие поддержки SIMD-инструкций.** На момент выполнения работы компилятор Go уже содержал встроенную поддержку векторных инструкций для нескольких архитектур. В частности, для x86-64 реализованы операции, использующие расширения SSE и AVX. Для ARM64 поддерживаются инструкции NEON. В стадии активной разработки находятся поддержка масштабируемых векторных расширений ARM SVE и векторных инструкций WebAssembly. Это означает, что компилятор уже располагает необходимым набором машинно-зависимых кодов операций для эмиссии векторного кода, и SLP-векторизатор может оперировать ими, не требуя кардинального расширения системы типов SSA. Наличие готовой SSA-инфраструктуры упрощает генерацию векторных инструкций в проходе, сводя задачу к корректному отображению скалярных групп на существующие векторные аналоги.

Эти свойства формируют благоприятную основу, но требуют аккуратного учёта при проектировании — в первую очередь в части работы с памятью.

5.6 Архитектурные ограничения для внедрения SLP

Несмотря на удобство SSA-представления, существующая архитектура компилятора Go накладывает ряд ограничений.

5.6.1 Явное моделирование памяти

Существенной особенностью SSA-представления в компиляторе Go является явное представление состояния памяти через специальный тип `TypeMem`. В отличие от классических SSA-реализаций, где операции доступа к памяти часто рассматриваются как побочные эффекты, в SSA компилятора Go память представлена как отдельная сущность в графе зависимостей данных.

Каждая операция чтения из памяти принимает текущее состояние памяти в качестве одного из аргументов. Операции записи формируют новое состояние памяти, которое затем используется последующими операциями. Например:

```
m0 = InitMem          // создание начального состояния памяти
v0 = ConstInt 0
v1 = Load ptr1, m0
m1 = Store ptr2, v0, m0 // изменение состояния памяти
v2 = Load ptr3, m1
```

Такое представление обеспечивает явное кодирование зависимостей по памяти в SSA-графе. Для реализации SLP-векторизации данная особенность имеет двоякое значение.

С одной стороны, наличие явных зависимостей по памяти упрощает проверку корректности преобразований, поскольку позволяет непосредственно анализировать порядок операций доступа к памяти. С другой стороны, объединение нескольких операций чтения или записи в одну векторную операцию требует строгого соблюдения семантики цепочки состояний памяти. Например, две операции загрузки могут быть объединены только в случае отсутствия промежуточных модификаций памяти, способных изменить наблюдаемое значение. Аналогично, при векторизации операций записи необходимо обеспечить консистентность состояний памяти.

Таким образом, работа с типом `TypeMem` является важной особенностью для реализации SLP-векторизации в компиляторе Go и требует специального учёта при анализе допустимости преобразований.

5.6.2 Ограниченная аналитическая инфраструктура

Компилятор Go не содержит многих анализов, на которые опираются векторизаторы LLVM:

- отсутствует `ScalarEvolution` — нет точного анализа выражений адресации и сравнения смещений в общем виде;
- отсутствует `AliasAnalysis` — невозможно определить, пересекаются ли области памяти двух произвольных указателей;
- нет `LoopInfo` — нельзя получить информацию о границах циклов и инвариантах.

Единственным доступным средством анализа смежности адресов является логика, реализованная в проходе `memcombine`. Она опирается на разбор указателей вида «база + индекс + смещение» и сравнение баз. SLP-векторизатор вынужден либо повторно использовать эту логику, либо реализовывать собственные упрощённые проверки.

5.6.3 Требования к времени компиляции

Одним из ключевых принципов разработки компилятора Go является сохранение высокой скорости сборки. Любой новый проход, в том числе векторизатор, обязан вносить минимальные накладные расходы. Это означает, что алгоритмы поиска и расширения групп должны быть ограничены по сложности (линейной или почти линейной относительно размера блока), а дорогостоящие рекурсивные обходы значительно ограничены. Таким образом, проектируемый векторизатор должен находить баланс между глубиной анализа и временем компиляции, сознательно отказываясь от ряда возможностей ради сохранения быстродействия.

5.6.4 Особенности рантайма языка

Рантайм языка Go накладывает дополнительные ограничения, которые необходимо учитывать при внедрении SIMD-операций в общий компиляторный пайплайн. В первую очередь это касается взаимодействия с сборщиком мусора и механизмом переключения горутин.

Сборщик мусора и указатели в SIMD-регистрах. Go использует точный сборщик мусора, который для своей работы требует в любой точке синхронизации (safe-point) полной информации о живых указателях на кучевые объекты. Традиционно такие указатели хранятся в регистрах общего назначения или на стеке, где компилятор может сгенерировать для них специальные карты. SIMD-регистры, как правило, не предназначены для хранения скалярных указателей и не поддерживают генерацию соответствующих метаданных в текущей инфраструктуре компилятора Go. Как следствие, векторизованный код не должен содержать указатели на Go-объекты внутри SIMD-регистров в моменты, где возможна остановка для сборки мусора. На практике это означает, что SLP-векторизатору безопасно работать только с типами, не содержащими указателей.

Планировщик горутин и сохранение SIMD-контекста. Рантайм Go осуществляет кооперативное и некооперативное переключения горутин, в ходе которых должен быть сохранён и восстановлен полный контекст исполнения, включая значения

всех регистров. Современные версии компилятора Go уже включают в сохраняемый фрейм горутины состояние SSE/AVX/NEON регистров (например, XMM, YMM на x86-64, V-регистры на ARM64 [?]), что добавляет небольшие, но ненулевые накладные расходы при каждом переключении. Это необходимо принимать во внимание при оценке общей эффективности векторизации. Однако для большинства вычислительно насыщенных участков, где векторизация приносит наибольший эффект, переключения горутин редки, и данными накладными расходами можно пренебречь.

Таким образом, особенности рантайма Go задают некоторые ограничения применимости SLP-векторизатора: обработка данных без указателей, ограничение векторизации между местами потенциального кооперативного вытеснения (например, вызовами функций).

5.7 Выбор точки интеграции

В рамках данной работы проход SLP-векторизации размещается непосредственно перед этапом `lower`, преобразующим машинно-независимую SSA-форму в машинно-зависимую.

Выбор данной точки интеграции обусловлен особенностями организации оптимизационного конвейера компилятора Go. К моменту выполнения прохода уже завершены основные SSA-оптимизации, включая локальные упрощения и устранение избыточных вычислений. Это позволяет анализировать относительно стабилизированную структуру вычислительного графа. Одновременно с этим программа всё ещё представлена в достаточно высокоуровневой SSA-форме, сохраняющей семантическую структуру операций и типов.

Размещение прохода до понижения SSA-формы к машинно-зависимой позволяет реализовать общий векторизатор, используя уже существующую инфраструктуру понижения обобщённых SIMD-операций в машинные инструкции целевой архитектуры.

5.8 Выводы по главе

В данной главе была рассмотрена архитектура SSA-бэкенда компилятора языка Go и проведён анализ её особенностей с точки зрения реализации автоматической SLP-векторизации.

Показано, что используемое в компиляторе SSA-представление предоставляет удобную основу для локального анализа зависимостей и выявления векторизуемых ациклических участков кода. Этому способствует явное представление вычислительных зависимостей между операциями, строгая типизация SSA-инструкций, а также использование специального типа `TypeMem`, обеспечивающего явное моделирование состояния памяти в графе вычислений.

Установлено, что наличие развиваемой инфраструктуры SIMD-операций на уровне SSA позволяет реализовывать автоматическую векторизацию без необходимости создания отдельного промежуточного представления для векторных вычислений.

Вместе с тем выявлен ряд архитектурных ограничений, влияющих на проектирование алгоритма. К ним относятся ограниченность существующей аналитической инфраструктуры и необходимость сохранения малого времени компиляции.

На основании проведённого анализа в качестве точки интеграции SLP-векторизирующего прохода выбран этап, непосредственно предшествующий этапу понижения до машинно-зависимой SSA-формы. Такое размещение позволяет выполнять анализ над высокоуровневым SSA-представлением и одновременно использовать существующий механизм последующего преобразования SIMD-операций в машинные инструкции целевой архитектуры.

Полученные результаты определяют основные требования к проектируемому алгоритму SLP-векторизации, рассматриваемому в следующей главе.

6 Проектирование SLP-векторизатора

В данной главе описывается архитектура спроектированного SLP-векторизатора, предназначенного для работы в составе компилятора Go. Изложение построено от общих требований и ограничений к деталям каждого этапа обработки.

6.1 Требования и ограничения для Go-векторизатора

Проектируемый векторизатор должен удовлетворять следующим требованиям, вытекающим из анализа компилятора Go, проведённого в главе ??.

- **Корректность при явных зависимостях по памяти.** В Go SSA операции с памятью связаны глобальной цепочкой состояний Memory. Любое преобразование обязано сохранять этот порядок. Векторизатор должен гарантировать, что объединение нескольких скалярных инструкций в одну векторную не нарушит модель памяти. Консервативный подход требует, чтобы упаковываемые инструкции либо разделяли общий входной Memory-аргумент, либо (для цепочек сохранений) были связаны последовательной зависимостью по памяти.
- **Ограничение времени компиляции.** Компилятор Go ориентирован на быструю сборку, поэтому векторизатор не должен добавлять существенных задержек.
- **Поддержка векторных инструкций ARM64 NEON.** Основной целевой архитектурой на текущем этапе является ARM64 с расширением NEON, предоставляющим 128-битные векторные регистры. Векторизатор должен генерировать операции над 128-битными векторами, составленными из двух 64-битных целочисленных элементов. Поддержка x86-64 SSE/AVX2 и других архитектур может быть добавлена позже без изменения алгоритмического ядра.
- **Обработка типов, не содержащих указателей.** Из-за требований точного сборщика мусора векторизации не подлежат указательные типы.

6.2 Общая архитектура

Векторизатор оформлен как проход, работающий на уровне целой функции. Он последовательно обходит базовые блоки функции и для каждого блока пытается выделить и векторизовать группы изоморфных независимых инструкций.

Ключевыми структурами данных являются:

- **pack** — упорядоченный набор изоморфных значений. Порядок внутри pack'a соответствует линиям будущего векторного регистра. Например, `pack{v1, v2}` объединяет две операции, которые станут элементами 0 и 1 векторного регистра. Далее для обозначения **pack** будет использоваться термин — группа.
- **packSet** — набор групп, накапливаемый в процессе анализа одного базового блока.

Общая схема работы для каждого базового блока включает следующие последовательные этапы:

1. **Поиск начальных пар.** Сканируются инструкции блока, отыскиваются пары смежных обращений к памяти, удовлетворяющих условиям изоморфизма, независимости и смежности адресов. На их основе формируются начальные пары.

2. **Расширение групп вверх и вниз.** От начальных пар итеративно достраиваются пары операций для операций-аргументов и операций-потребителей. Процесс продолжается до тех пор, пока удаётся добавлять новые пары. На этом этапе образуется граф групп, отражающий потенциально векторизуемое дерево выражений.
3. **Комбинирование.** Соседние группы, конец одной из которых совпадает с началом другой, сливаются в более широкие операции. Например, `pack{v1, v2}` и `pack{v2, v3}` превращаются в `pack{v1, v2, v3}`. Это позволяет формировать векторы, ширина которых превышает два элемента, если исходный код содержит большие последовательности однотипных операций.
4. **Разбиение на компоненты связности.** Группы, связанные через аргументы, объединяются в независимые компоненты (подграфы). В дальнейшем каждая компонента обрабатывается отдельно. Это позволяет не отбрасывать сразу все компоненты, если некоторые из них не пройдут дальнейшую проверку.
5. **Валидация.** Для каждой выделенной компоненты проверяется ряд условий: отсутствие недопустимых внешних использований, замкнутость операндов и когерентность линий. Непрошедшие проверку компоненты отбрасываются.
6. **Планирование.** Для прошедшей валидацию компоненты операции упорядочиваются топологически так, чтобы ни одна группа не находилась в выходном наборе раньше тех, от которых она зависит по данным.
7. **Легализация.** Группы, длина которых превышает аппаратную ширину векторного регистра (2 элемента для 128-битных векторов при 64-битном скалярном типе), разбиваются на подгруппы.
8. **Генерация кода.** Выполняется эмиссия векторных SSA-инструкций: для каждой группы создаётся одна векторная инструкция, заменяющая собой исходные скалярные. Внутренние связи между группами заменяются прямыми ссылками на результаты векторных инструкций. Для значений, используемых вне векторизованного фрагмента, вставляются инструкции извлечения отдельных элементов.

Далее каждый этап рассматривается подробно.

6.3 Поиск кандидатов для векторизации

Начальный этап нацелен на выделение пар инструкций, которые послужат основой для дальнейшего расширения. В качестве начальных инструкций служат обращения к памяти — загрузки и сохранения. Этот выбор мотивирован тем, что смежные обращения к памяти предоставляют естественный параллелизм.

Для пары инструкций (v_1, v_2) проверяется следующий набор условий:

- **Изоморфизм:** v_1 и v_2 имеют одинаковый код операции и совместимый тип результата.
- **Векторизуемость операции:** операция входит в «белый список» разрешённых для векторизации операций.
- **Смежность для обращений к памяти:** для инструкций обращения к памяти их адреса должны быть смежными и упорядоченными. Проверка выполняется процедурой, которая использует инфраструктуру прохода `memcombine`.

- **Отсутствие конфликтов с уже существующими группами:** ни v_1 , ни v_2 не должны входить в уже сформированные группы.
- **Независимость:** проверка, что v_1 и v_2 не зависят друг от друга по данным и разделяют общее состояние памяти. Только операции сохранения являются исключением при проверке общего состояния памяти, так как они обязательно будут иметь разные состояния. Для них проверяется, образуют ли они последовательную Мемогу-цепочку.

Дополнительно исключаются инструкции с побочными эффектами. После нахождения пары создаётся новая группа размера 2, которая добавляется в общий набор.

6.4 Построение графа зависимостей и расширение групп

После формирования начальных пар алгоритм итеративно расширяет множество групп, двигаясь по потоку данных. Процесс выполняется до сходимости: за одну итерацию последовательно вызываются процедуры расширения вниз и вверх для каждой группы. Если в результате появляются новые группы, итерация повторяется.

Расширение вниз. Для группы $p = \{v_1, v_2\}$ анализируются все инструкции-потребители результатов v_1 и v_2 в пределах того же базового блока. Для каждой пары потребителей (t_1, t_2) проверяются условия упаковки. Выбранная пара добавляется в набор как новая группа.

Расширение вверх. Для того же $p = \{v_1, v_2\}$ перебираются все аргументы v_1 и v_2 . Применяются те же критерии изоморфизма и независимости.

После завершения расширения накопленные группы могут быть объединены в более длинные. Процедура совмещения просматривает все пары групп: если конец одной совпадает с началом другой (например, $\text{pack}\{v_1, v_2\}$ и $\text{pack}\{v_2, v_3\}$), они сливаются в одну группу $\text{pack}\{v_1, v_2, v_3\}$. Процедура повторяется до тех пор, пока находятся новые объединения.

Следующим шагом набор групп разбивается на компоненты связности по графу, в котором вершины — это группы, а ребро существует, если одна группа использует в качестве аргумента значение из другой группы. Каждая компонента затем обрабатывается независимо, что позволяет векторизовать в одном блоке несколько не связанных друг с другом групп.

6.5 Валидация и планирование

Перед генерацией кода каждая выделенная компонента должна пройти ряд проверок корректности.

6.5.1 Проверка внешних использований

Для каждого значения, входящего в какую-либо группу, отыскиваются все его использования в функции. Использование считается допустимым, если оно принадлежит одной из групп той же компоненты. Также для операций сохранения внешнее использование порождённого ими значения Мемогу разрешено, так как оно лишь продолжает глобальную цепочку памяти.

Существует небольшая эвристика, разрешающая ограниченное количество внешних использований, если это будет считаться выгодным. На текущий момент для наборов, состоящих из более чем трёх групп, разрешено иметь до четырёх внешних использований. Любое другое внешнее использование приводит к отбрасыванию всей компоненты. Это консервативное решение исключает необходимость дорогостоящей скаляризации для внешних пользователей.

6.5.2 Проверка замкнутости операндов

У каждого значения в группе анализируются аргументы. Аргументы, не являющиеся указателями и не относящиеся к типу Мемогу, обязаны принадлежать какой-либо группе той же компоненты. Это гарантирует, что все внутренние зависимости остаются внутри векторизуемого подграфа.

6.5.3 Проверка когерентности линий

На данном этапе проверяется, что все группы в компоненте имеют одинаковую длину. Также требуется, чтобы аргументы каждой инструкции происходили из одной и той же линии соответствующего аргумента.

6.5.4 Проверка исходных аргументов

TODO: Описание проверки исходных аргументов (например, констант, указателей) будет добавлено после уточнения деталей реализации.

После успешной валидации выполняется планирование. Цель — выстроить группы в порядке, гарантирующем, что к моменту генерации кода для данной группы все её аргументы уже вычислены.

6.6 Легализация и генерация кода

Легализация. Полученный упорядоченный набор может содержать группы, длина которых превышает 2 — аппаратную ширину 128-битного векторного регистра для 64-битных элементов. Процедура разбивает каждую такую группу на подгруппы по 2 элемента. Например, `pack{v1, v2, v3, v4}` превращается в две подгруппы `{v1, v2}` и `{v3, v4}`. Если длина группы не кратна 2, легализация считается неуспешной, и компонента не векторизуется.

Генерация векторного кода. Для каждой группы создаётся одна векторная SSA-инструкция. Если аргумент исходной скалярной инструкции принадлежит другой группе, он заменяется на соответствующее векторное значение. Если нет — например, для указателей и памяти — аргумент передаётся как есть.

После создания всех векторных инструкций выполняется замена скалярных использований:

- Внутренние использования в других группах уже учтены на этапе формирования аргументов.
- Для сохранений обновляется аргумент Мемогу у последующих инструкций: вместо исходного значения памяти, порождённого скалярным сохранением, подставляется память, возвращённая векторным сохранением.
- Для прочих внешних использований вставляются соответствующие инструкции, извлекающие отдельные элементы из векторного регистра.

После этих замен исходные скалярные инструкции становятся «мёртвым кодом» и удаляются.

6.7 Принятые компромиссы и направления развития

При проектировании был сделан ряд осознанных упрощений, позволивших получить работоспособный векторизатор в условиях ограниченной инфраструктуры Go:

- **Поддерживаемые типы и архитектура.** Только 128-битные векторы. Этого достаточно для демонстрации принципиальной работоспособности и ускорения ряда вычислительных шаблонов; расширение на 256-битные векторы является технической задачей, не требующей изменения алгоритмического ядра.
- **Отсутствие модели стоимости.** Это упрощает код и снижает накладные расходы, но может приводить к неоптимальным решениям в случаях, когда затраты на упаковку/распаковку превышают выигрыш от SIMD-операции. Для первой консервативной реализации можно обойтись без модели стоимости.
- **Консервативная модель памяти.** Требование единого Memory-аргумента или строгой цепочки для сохранений исключает многие потенциально векторизуемые случаи, где между двумя независимыми загрузками находится запись по непересекающемуся адресу. Преодоление этого ограничения потребовало бы внедрения более сложного анализа.
- **Отсутствие инструкций перестановки элементов в векторе (shuffle).** Это означает, что ситуации, требующие перестановки элементов вектора (например, если одна линия использует аргумент из другой линии), не поддерживаются.

Дальнейшие направления развития включают:

- введение упрощённой модели стоимости, оценивающей количество вставок/извлечений, которая позволила бы более агрессивно векторизовывать код;
- улучшение анализа памяти для распознавания независимых загрузок с разными Memory-аргументами;
- реализацию базовых **shuffle**-операций для поддержки некогерентных линий;
- векторизация редукций.

Перечисленные компромиссы и направления формируют реалистичный план развития, сохраняя текущую реализацию компактной и пригодной для включения в основной компилятор Go.

7 Реализация

В данной главе описывается практическая реализация SLP-векторизатора в составе компилятора Go. Рассматриваются среда разработки, ключевые структуры данных, алгоритмы каждого этапа, генерация векторного кода и методика тестирования.

7.1 Инструментарий и среда разработки

Разработка велась во внутреннем форке языка Go (согласованном с версией go1.22) с целевой архитектурой ARM64 с поддержкой SIMD-расширения NEON. Проход SLP реализован в пакете `cmd/compile/internal/ssa` в виде отдельного файла `slp.go`. Новый проход зарегистрирован в общем списке оптимизаций компилятора.

Для отладки использовалась встроенная в компилятор система флагов: переменная `debug`, управляемая флагом `-d=ssa/slp/debug=N`, а также переменная окружения `GOSSAFUNC`, позволяющая вывести SSA-граф конкретной функции до и после прохода. Это позволило визуально контролировать корректность преобразований.

Тестирование выполнялось с помощью системы `go test`, включая специализированные тесты `codegen`: `asmcheck` (проверка наличия/отсутствия ожидаемых векторных инструкций в ассемблерном листинге) и `errorcheck` (проверка диагностических сообщений для случаев, когда векторизация не должна применяться).

7.2 Базовые структуры данных и проверки

Основными структурами данных являются типы `pack` и `packSet`.

`type pack []*Value`

`pack` определён как упорядоченный срез указателей на значения SSA. Все значения в одном `pack`'е изоморфны и упорядочены согласно линиям.

`type packSet []pack`

`packSet` — срез `pack`'ов, накапливаемый в процессе анализа блока.

Ключевые вспомогательные проверки, используемые на этапе анализа:

- `isIsomorphic(v1, v2)` — проверяет, что две инструкции имеют одинаковый код операции и одинаковый тип результата. Дополнительно требует равенства количества аргументов.
- `isIndependent(v1, v2)` — проверяет, что инструкции не зависят друг от друга по данным и разделяют одно состояние памяти.
- `isAdjacent(v1, v2)` — проверяет смежность двух обращений к памяти. Использует анализ из прохода `memcombine` для разложения указателя на базовую часть, индекс и смещение. Два обращения считаются смежными и упорядоченными, если у них совпадают база и индекс, а разность константных смещений равна размеру элемента.
- `canPack(v1, v2, Ps, b)` — собирает все проверки для возможности образования новой пары: изоморфизм, векторизуемость операции и типа, смежность (для памяти), отсутствие конфликтов с существующими группами (ни одно значение уже не задействовано) и независимость.

7.3 Проход `slp`: главный управляющий цикл

Входная точка прохода — функция `slp(f *Func)`. Она обходит все базовые блоки функции; для каждого блока b последовательно выполняются следующие шаги:

1. **Инициализация** (`findSeeds`): формируется начальный набор из пар смежных загрузок/сохранений.
2. **Расширение** (`extendPackSet`): итеративно вызываются `followUse` и `followDef` для добавления зависимых инструкций.
3. **Комбинирование** (`combinePacks`): соседние группы сливаются.
4. **Разбиение** (`splitPackSets`): набор групп разбивается на связные компоненты.
5. Для каждой компоненты:
 - валидация (`checkPackSet`) — проверка корректности;
 - планирование (`schedulePacks`) — топологическая сортировка;
 - легализация (`legalizePackSet`) — приведение к аппаратной ширине 16 байт;
 - генерация кода (`emitPacks`) — создание векторных инструкций и замена скаляров.

При наличии нескольких компонент каждая обрабатывается независимо, что позволяет векторизовать несколько не связанных участков в одном блоке.

7.4 Этапы анализа

7.4.1 Поиск начальных пар

Процедура двойным вложенным циклом перебирает все значения блока. Для каждой пары (v_1, v_2) , где обе инструкции являются обращениями к памяти (`OpLoad` или `OpStore`), вызывается `canPack`. При успехе создаётся новая группа. Сложность этого шага квадратична от числа инструкций в блоке, но на практике размеры блоков невелики, и с учётом ранних отсечений (проверка кода операции и типа до вызова `canPack`) накладные расходы остаются приемлемыми.

7.4.2 Расширение групп

Выполняется итеративный цикл до сходимости: за каждую итерацию для всех текущих групп вызываются `followUse` и `followDef`.

- `followDef` для группы `pack{v1, v2}` перебирает все аргументы; для каждого проверяет `canPack(v1.Args[i], v2.Args[i])` и добавляет новые группы при успехе. Добавление происходит для всех совместимых пар аргументов.
- `followUse` работает аналогично: он находит все использования v_1 и v_2 внутри текущего блока и проверяет, можно ли их сгруппировать.

7.4.3 Комбинирование

После расширения набор групп содержит только пары. Процедура ищет группы, такие что конец одной совпадает с началом другой, и объединяет их. Исходные мелкие группы заменяются объединённой. Процесс повторяется, пока возможны слияния.

7.4.4 Разбиение на компоненты связности

Алгоритм выделяет связанные компоненты графа, вершинами которого являются группы, а ребро между ними существует, если какой-либо аргумент значения из одной группы принадлежит другой группе, или наоборот. Это даёт набор независимых компонент.

7.5 Валидация и планирование

7.5.1 Валидация

Выполняет три основные проверки:

1. **checkExternalUses** — для каждого значения в группах ищутся все его использования в функции (**findAllUses**). Использование в том же наборе, а также особые случаи (**Store**-память для продолжения цепочки памяти) считаются допустимыми. Также для длинных цепочек разрешено ограниченное количество внешних использований. Любое другое внешнее использование приводит к отказу от векторизации всей компоненты.
2. **checkClosedOperands** — проверяет, что все аргументы принадлежат тому же набору групп. Нарушение замкнутости свидетельствует о зависимости, выходящей за пределы векторизуемого подграфа.
3. **checkLaneCoherence** — удостоверяется, что все группы в компоненте имеют одинаковую длину и что значения используются в тех же линиях, в которых они производятся. Это проверка когерентности линий; в противном случае потребовались бы **shuffle**-инструкции.

7.5.2 Планирование

Реализована простая топологическая сортировка. Для каждой группы рассматриваются аргументы её первого элемента (поскольку все элементы изоморфны, структура аргументов одинакова). Пропуская указатели и память, алгоритм находит группу-аргумент. Если такая группа ещё не отмечена запланированной, текущая группа откладывается. Итерации продолжаются, пока все группы не будут запланированы.

7.6 Генерация векторного SSA-кода

7.6.1 Легализация

Перед генерацией кода группы разбиваются до ширины в 128 бит. Если ширина групп не кратна 128 бит, то весь набор отбрасывается.

7.6.2 Создание векторных инструкций

Для каждой группы создаётся ровно одна векторная инструкция. Для отображения используется глобальная хеш-таблица соответствия. Примеры:

- **OpLoad** → **OpLoad** с типом результата **TypeVec128**.
- **OpStore** → **OpStore** с типом результата **TypeVec128**.
- **OpSub64** → **OpSubInt64x2** (векторное вычитание).
- **OpCpy** → **OpCpy** с типом **TypeVec128**.

Для операций обращений к памяти адрес нагрузки берётся из первого элемента группы (левая граница), память — с помощью метода `getMemoryArg`, возвращающего общий `Memory` для всей группы. Для операций, использующих другие группы, нужные аргументы находятся в кэше уже созданных векторных операций.

Все вновь созданные векторные значения сохраняются в `packToSimd` (отображение `pack` $\rightarrow$ `Value`), который служит кэшем для последующих групп.

7.6.3 Замена скалярных использований

После создания всех векторных инструкций итеративно обходятся исходные группы и для каждого значения обновляются его использования:

- Если использование принадлежит другой группе (т.е. внутреннее), оно уже было подставлено на этапе формирования аргументов векторной инструкции.
- Для значений `Store` обновляется аргумент памяти у следующих за ними инструкций: вместо результата скалярного `Store` подставляется память от векторного `Store`.
- Для внешних использований вставляется `OpGetElem{Type}x{Number of lanes}` — инструкция извлечения элемента из векторного регистра по индексу, соответствующему позиции значения в группе.

После замены исходные скалярные инструкции теряют всех пользователей и удаляются следующим проходом `deadcode`.

7.7 Тестирование корректности

Тестирование проводилось на двух уровнях:

- **errorcheck-тесты** — файлы с директивами `// errorcheck`, проверяющие, что SLP-проход не применяется в известных сложных случаях. В теле функций размещены аннотации вида `// ERROR "текст диагностики"`. При запуске компилятора с флагом `-d=ssa/slp/debug=1` векторизатор выводит сообщения с причиной отказа, и тестовая система сопоставляет их с ожидаемыми. Таким образом проверяются сценарии: разные состояния памяти, вызов между инструкциями, внешние использования, нечётная ширина и др.
- **asmcheck-тесты** — проверяют наличие ожидаемых векторных инструкций в ассемблерном выводе для ARM64. В комментариях к функциям указаны регулярные выражения (например, `// arm64:\VSub\t`), которые должны присутствовать в сгенерированном коде.

Тесты обеспечивают регрессионную защиту при дальнейшем развитии прохода и документируют его текущие возможности и ограничения. Ручное тестирование с помощью `GOSSAFUNC` позволяло визуально инспектировать SSA-граф до и после прохода, отслеживая корректность подстановок и отсутствие оборванных зависимостей.

8 Экспериментальная оценка реализации

8.1 Цели и методика оценки

Целью экспериментального исследования является оценка эффективности реализованного прохода SLP-векторизации. В рамках оценки решаются следующие задачи:

- измерение ускорения времени выполнения программ, получаемого за счёт применения предложенной оптимизации;
- оценка накладных расходов на этапе компиляции;
- сравнение результатов с базовой версией компилятора Go без реализованного прохода;
- сопоставление качества векторизации с компилятором `gollvm`.

Экспериментальная методика основана на сравнении трёх конфигураций компиляции:

1. стандартный компилятор Go;
2. модифицированная версия компилятора с реализованным проходом SLP-векторизации;
3. компилятор `gollvm`.

Для измерения времени выполнения использовался встроенный пакет тестирования [?], а для комплексных нагрузок применялись внешние тестовые наборы. Каждый эксперимент выполнялся многократно, после чего применялась утилита `benchstat` [?], предназначенная для статистической обработки результатов. Данная утилита позволяет агрегировать результаты многократных запусков, вычислять медианные значения, рассчитывать погрешность и оценивать статистическую значимость различий.

Для уменьшения влияния фоновой активности измерения проводились на изолированном сервере, специально предназначенном для замеров производительности.

8.2 Выбор тестовых наборов

Для оценки эффективности реализации был выбран набор тестов, включающий как синтетические микробенчмарки, так и реальные приложения.

8.2.1 Набор синтетических тестов

Синтетические тесты предназначены для оценки эффективности векторизации в контролируемых условиях. Они включают:

- арифметические операции над структурами, содержащими поля типа `int64`;
- аналогичные операции для `float64`;
- операции над структурами с полями типа `int32`.

Каждый тест представляет собой последовательность однотипных независимых операций, потенциально пригодных для объединения в SIMD-инструкции. Такой набор позволяет изолированно оценить качество поиска SLP-графов и эффективность генерации векторных инструкций.

8.2.2 Трассировщик лучей

Для оценки поведения оптимизации в вычислительно насыщенном приложении использовался простой трассировщик лучей, реализованный в проекте `oneweekend` [?], основанном на материалах книги *Ray Tracing in One Weekend* [?]. Данное приложение активно использует векторную арифметику над типом `Vec3`.

Поскольку реализованный проход ориентирован на SIMD-регистры фиксированной ширины, структура `Vec3`, содержащая три значения типа `float64`, была модифицирована путём добавления четвёртой компоненты. Это позволило представить операции над векторами в виде SIMD-операций без необходимости частичного заполнения регистра. Этот бенчмарк позволяет оценить эффективность подхода в условиях, близких к реальным задачам компьютерной графики.

8.2.3 etcd

В качестве примера промышленного приложения использовался проект `etcd` [?]. Выбор обусловлен следующими причинами:

- значительный объём кода;
- наличие вычислительно интенсивных участков;
- практическая значимость.

Использование `etcd` позволяет оценить влияние оптимизации на крупные реальные приложения, где потенциальные возможности векторизации ограничены сложной структурой программы.

8.2.4 Sweet

Для комплексной оценки применялся набор `Sweet benchmarks` [?], используемый разработчиками компилятора Go для оценки производительности изменений. Этот набор включает как тесты времени выполнения, так и сценарии измерения времени сборки, что делает его особенно полезным для анализа компромисса между качеством оптимизации и накладными расходами компиляции.

8.3 Результаты измерения производительности

8.3.1 Набор синтетических тестов

На микробенчмарках наблюдается наибольший эффект, поскольку структура вычислений соответствует предположениям алгоритма SLP-векторизации.

Таблица 1: Ускорение микробенчмарков

Тест	Базовое время (нс)	Ускорение (раз)
Int64Arith	TODO	TODO
Float64Arith	TODO	TODO
Int32Arith	TODO	TODO

Полученные результаты демонстрируют ускорение до X%, что подтверждает корректность поиска векторизуемых групп операций и генерации SIMD-инструкций.

8.3.2 Трассировщик лучей

На трассировщике лучей достигнуто ускорение $X\%$. Эффект обусловлен интенсивным использованием операций над векторами и высокой плотностью арифметических вычислений.

8.3.3 etcd

Для `etcd` ускорение носит менее выраженный характер. Это объясняется:

- меньшей долей вычислений, пригодных для SLP-векторизации;
- ограничениями текущего анализа зависимостей;
- высокой долей операций управления и обращений к памяти.

8.3.4 Sweet

TODO: Вставить результаты измерений на наборе Sweet.

8.4 Влияние на время компиляции

TODO: Описать измерение времени компиляции для различных тестов. Сравнить базовый компилятор и модифицированную версию. Ожидается незначительное увеличение (единицы процентов).

8.5 Сравнение с `gollvm`

TODO: Сравнить качество векторизации и производительность сгенерированного кода для выбранных тестов. Отметить, что `gollvm` может давать большее ускорение за счёт более агрессивных оптимизаций, но цена — значительное увеличение времени компиляции.

8.6 Обсуждение ограничений и не векторизованных случаев

В ходе экспериментов выявлены сценарии, в которых предложенный SLP-векторизатор не срабатывает или даёт незначительный выигрыш:

- наличие операций с памятью, разделяющих разные состояния Memory;
- использование указателей внутри векторизуемых структур;
- неполная ширина группы (не кратна 2);
- внешние использования промежуточных результатов.

Эти ограничения связаны с принятыми компромиссами (см. раздел 6.7) и могут быть преодолены в будущих версиях.

8.7 Направления будущей работы

На основе полученных результатов можно выделить следующие направления дальнейшего развития:

- внедрение упрощённой модели стоимости для более агрессивной векторизации;
- улучшение анализа памяти для распознавания независимых загрузок;
- поддержка `shuffle`-инструкций для некогерентных линий;
- расширение на другие архитектуры (x86-64, SVE);
- векторизация редукций и частичных циклов.

9 Заключение

В ходе выполнения дипломной работы были решены все поставленные задачи.

1. Проведён анализ существующих методов автоматической векторизации, в частности подходов цикловой и SLP-векторизации. Выявлены преимущества и ограничения каждого метода, обоснован выбор SLP-подхода как наиболее подходящего для интеграции в компилятор Go.
2. Исследована архитектура оптимизационного конвейера компилятора Go, включая SSA-представление, систему оптимизирующих проходов и особенности моделирования памяти через тип `TypeMem`. Установлены ключевые ограничения: отсутствие сложных анализов (псевдонимов, эволюции индуктивных переменных), требование быстрой компиляции и специфика рантайма (точный сборщик мусора, планировщик горутин).
3. Разработан алгоритм выявления векторизуемых групп операций, включающий этапы поиска начальных пар смежных обращений к памяти, расширения групп по потоку данных (вверх и вниз), комбинирования, разбиения на компоненты связности, валидации (внешние использования, замкнутость операндов, когерентность линий) и планирования.
4. Реализован прототип SLP-векторизатора в составе компилятора Go (пакет `cmd/compile/internal/slp.go`). Выполнена генерация 128-битных векторных инструкций для архитектуры ARM64 (NEON), реализована замена скалярных операций векторными с корректным обновлением цепочек памяти и вставкой извлечений для внешних использований.
5. Проведена экспериментальная оценка разработанного решения на синтетических тестах, трассировщике лучей, промышленном приложении `etcd` и наборе `Sweet`. Подтверждена корректность преобразований и достигнуто ускорение до XX% на вычислительно интенсивных фрагментах при незначительном увеличении времени компиляции (менее 1%). Выявлены сценарии, где векторизация не применяется (различные состояния памяти, указатели в структурах, неполная ширина группы), что соответствует принятым компромиссам.

Практическая значимость работы состоит в создании действующего прототипа SLP-векторизатора для компилятора Go, который:

- демонстрирует принципиальную возможность интеграции SIMD-оптимизаций в стандартный компилятор;
- может служить основой для дальнейшего развития оптимизирующих возможностей `cmd/compile`;
- предоставляет базу для расширения на другие архитектуры (x86-64, SVE) и поддержки более сложных векторизуемых шаблонов.

Дальнейшие направления развития включают:

- введение упрощённой модели стоимости для более агрессивной векторизации;
- улучшение анализа памяти для распознавания независимых загрузок с разными `Memory`-аргументами;

- реализацию **shuffle**-инструкций для поддержки некогерентных линий;
- векторизацию редукций и частичных циклов.

Таким образом, цель работы — разработка и реализация прототипа механизма автоматической SLP-векторизации ациклических участков кода в компиляторе Go — достигнута. Полученные результаты подтверждают практическую пользу предложенного подхода и открывают путь к внедрению SIMD-оптимизаций в основной компилятор языка Go.

Список литературы

- [1] *Samuel Larsen, Saman Amarasinghe. TExploiting Superword Level Parallelism with Multimedia Instruction Sets / Saman Amarasinghe Samuel Larsen // Cambridge, MA 02139. — 2000.*
- [2] *Морз, Р. Бесстолкновительный PIC-метод / Р. Морз // Вычислительные методы в физике плазмы / Ed. by Б. Олдера, С. Фернбаха, М. Ротенберга. — М.: Мир, 1974.*
- [3] *Киселёв, А. А. Численное моделирование захвата ионов бесстолкновительной плазмы электрическим полем поглощающей сферы / А. А. Киселёв, Долгоносков М. С., Красовский В. Л. // Девятая ежегодная конференция «Физика плазмы в Солнечной системе». — 2014.*